

Final Report: Deployable UAN for One-Leg Sim-to-Real Actuator Correction

Unsupervised Actuator Network with Runtime-Available Observations

Khunanon Sawetkhotchakul 6606

Waratanut Kitkrongkajon 6671

Pawaris Tangtrakul 6674

FRA 503: Deep Reinforcement Learning, 2026

May 24, 2026

Abstract

This project investigates whether an Unsupervised Actuator Network (UAN) can reduce the sim-to-real gap of a low-cost legged-robot actuator using deployable simulator-side inputs. The policy receives command tracking error, simulated joint state, command change, and previous residual action. Real hardware data is used as the transition target during training and evaluation, but the policy input contains only information that would be available inside a future walking simulator. Experiments on the deployable evaluation set show that UAN reduces joint-position replay error on walk-air and walk-contact data. The final walk-air evaluation reduces q RMSE from 0.02673 rad to 0.01129 rad, and the final walk-contact evaluation reduces q RMSE from 0.02803 rad to 0.01281 rad. However, velocity RMSE generally increases and the sample-efficiency result is non-monotonic. The correct final claim is that the deployable UAN is a practical proof of concept for actuator position correction, but not yet a complete velocity/contact dynamics model.

1 Project Overview and Scope

The goal of the project is to reduce the gap between simulated and real actuator behavior for legged robots. Standard simulators often assume that the actuator follows the commanded joint target with idealized dynamics. Real motors and transmissions include friction, stiction, backlash, latency, compliance, and load-dependent behavior. These effects can make a locomotion policy that works in simulation fail on hardware.

The original proposal targeted a full hexapod, but the implemented final scope is a 3-DOF one-leg test rig. This scope is appropriate because the actuator mismatch can be measured directly without the extra complexity of body dynamics, gait stability, multi-leg coordination, and terrain interaction. The actuated joints are hip, knee, and ankle. The final objective is not full hexapod walking; it is to learn a residual actuator correction that makes simulated one-leg joint transitions closer to recorded hardware transitions.

The final code and most deployable evaluation results used in this report are in:

`New_physicalAI.2026-master/logs/rsl_rl/one_leg_uan_deploy/eval_drl_experiments.`

The updated final-policy Exp3 and Exp5 plots are taken from:

`physicalAI.2026-master.final/logs/rsl_rl/one_leg_uan_deploy/eval_drl_experiments.`

The most important code files are:

- `one_leg_uan_deploy.env.py`: deployable-observation UAN environment.
- `one_leg_uan_deploy.env.cfg.py`: simulation, observation, dataset, and reward configuration.
- `train_uan_deploy.py`: RSL-RL training entry point for `oneleg-uan-deploy`.
- `evaluate_deploy_dataset.py`: evaluation script for deployable UAN checkpoints.
- `uan/dataset.py`: hardware CSV loader and transition builder.

1.1 System Pipeline

The implemented workflow is a closed loop between recorded hardware motion and replay simulation. First, the physical one-leg rig is driven using predefined position commands and the measured joint states are logged. Second, the dataset loader converts each log into transition data containing current state, command, and next measured state. Third, Isaac Lab replays the command in simulation while the UAN policy adds residual effort. Finally, the simulated next state is compared with the recorded hardware next state and the transition error becomes the reinforcement learning reward.

Table 1: Deployable UAN system pipeline.

Stage	Input/Output	Purpose
Hardware logging	command, $q, \dot{q}$	Capture measured actuator response under multiple trajectory families.
Dataset loader	CSV $\rightarrow$ transition tensors	Validate columns, sort samples by time, and build replayable transitions.
Replay environment	command + simulated state	Run the one-leg simulator from recorded starts.
UAN policy	simulator-side observation $\rightarrow$ residual action	Add learned residual effort to reduce transition error.
Evaluation	baseline and UAN traces	Quantify q RMSE, dq RMSE, residual action, and time-series behavior.

This pipeline keeps the learning problem focused on actuator correction rather than full locomotion. The one-leg scope is an experimental control: body motion, multi-leg coordination, and terrain strategy are removed so that the measured result is mainly about whether the learned residual can make simulated hip, knee, and ankle transitions closer to hardware. A successful result here does not prove full hexapod walking, but it gives a useful actuator layer that can later be inserted into a walking-policy training pipeline.

1.2 DRL Framing

The UAN is trained with PPO, but it is not a walking controller. It is a residual dynamics policy. The action is a three-dimensional correction applied to the simulated actuator effort, and the reward is computed from one-step agreement with measured hardware. This makes the task different from normal locomotion reward design. There is no forward velocity term, gait style reward, energy reward for walking, or fall penalty. The goal is simply to make the simulator transition more realistic for the measured command sequence.

This framing is useful because torque labels are not required. A supervised actuator model would need target torques or a direct residual label, which are difficult to obtain accurately on a low-cost leg rig. The UAN instead learns from the consequence of its action: if the residual effort makes the next simulated state closer to the next hardware state, the reward improves. This is why the project still fits the DRL requirement while remaining focused on sim-to-real actuator modeling.

2 Deployable Observation Design

The deployable UAN is designed so that the policy can run using only simulator-side signals. For each timestep, the policy observes

$$x_t = [q_t^{cmd} - q_t^{sim}, q_t^{sim}, \dot{q}_t^{sim}, q_t^{cmd} - q_{t-1}^{cmd}, a_{t-1}].$$

Each term has three dimensions for the hip, knee, and ankle joints. Therefore the per-step observation size is

$$3 + 3 + 3 + 3 + 3 = 15.$$

With history length H , the full observation size is $15H$. For the default $H = 20$, the policy observation has 300 dimensions. The history window gives the network short-term context about tracking error, velocity, command changes, and its own recent residual effort.

Table 2: Deployable UAN observation terms.

Observation term	Meaning	Dimension
$q_t^{cmd} - q_t^{sim}$	Command tracking error in the simulator	3
q_t^{sim}	Current simulated joint position	3
$\dot{q}_t^{sim}$	Current simulated joint velocity	3
$q_t^{cmd} - q_{t-1}^{cmd}$	Command change between adjacent timesteps	3
a_{t-1}	Previous residual action produced by UAN	3

Each observation term has a specific role. The command tracking error tells the policy whether the simulated joint is currently lagging behind or overshooting the commanded target. The simulated position and velocity give the policy the dynamic state needed to avoid applying the same residual at every point in the motion. The command change term exposes whether the requested motion is accelerating, decelerating, or holding steady. The previous action gives the policy a short memory of its own intervention, which helps reduce discontinuous residual behavior when the history window is long enough.

The important design constraint is that the policy input must be available at runtime. During future walking-policy training, the simulator knows the command, simulated joint state, and previous residual action. It does not need a paired hardware replay trajectory to produce an action. Therefore, the trained UAN can be used as a learned actuator correction module inside a simulator rather than only as an offline analysis tool.

3 Method and Reward

The UAN policy outputs a normalized residual action

$$a_t = \pi_\theta(o_t), \quad a_t \in [-1, 1]^3.$$

The residual effort applied in simulation is

$$\tau_t^{res} = 2.8a_t.$$

The recorded hardware position command remains the nominal joint-position target. The UAN only adds residual effort to make the simulated transition closer to the measured hardware transition.

Although the policy no longer observes real data, the reward still compares the simulated next state against the recorded hardware next state:

$$r_t = 4.0 \exp\left(-\left\|\frac{q_{t+1}^{real} - q_{t+1}^{sim}}{0.05}\right\|^2\right) + 0.5 \exp\left(-\left\|\frac{\dot{q}_{t+1}^{real} - \dot{q}_{t+1}^{sim}}{0.5}\right\|^2\right) - 0.02\|a_t - a_{t-1}\|^2 - 0.001\|a_t\|^2.$$

This is the key distinction: real data is allowed as a training target, but not as a policy input. This makes the model usable later as an actuator correction layer inside a simulator.

Table 3: Reward terms and their expected effect.

Term	Weight/scale	Intended behavior
Position match	4.0, scale 0.05 rad	Strongly reward one-step joint-position agreement with hardware.
Velocity match	0.5, scale 0.5 rad/s	Encourage dynamic consistency without dominating the objective.
Action rate penalty	0.02	Discourage sudden changes in residual effort.
Action magnitude penalty	0.001	Prevent the residual policy from using unnecessarily large effort.

The reward weights explain a major pattern in the final results. Position matching is the dominant term, so the trained UAN reliably improves q RMSE. Velocity matching has a smaller weight and uses measured velocity, which may be noisier than position. Therefore, dq RMSE is treated as an important diagnostic rather than the only success criterion. A useful final model should eventually improve both position and velocity, but this project shows that position correction is currently the stronger and more stable outcome.

The zero-residual baseline is essential for interpreting the result. It uses the same simulator, same command replay, same initial states, and same evaluation windows, but sets the residual action to zero. Therefore, any improvement over this baseline is caused by the learned residual correction rather than a different trajectory or evaluation protocol. The baseline is not expected to have zero error, because the point of the project is that the simulator actuator dynamics do not perfectly match hardware.

4 Simulation and Training Setup

The deployable UAN uses Isaac Lab and RSL-RL PPO. The environment task is `oneleg-uan-deploy`. The simulator replays recorded hardware commands, applies the UAN residual effort, and scores the transition against recorded q_{next} and $\dot{q}_{next}$.

Table 4: Deployable UAN simulation and PPO setup.

Category	Parameter	Value
Simulator	engine/task	Isaac Lab, <code>oneleg-uan-deploy</code>
Simulator	timestep	0.005 s
Simulator	decimation	2
Robot	actuated joints	hip, knee, ankle
Training	parallel environments	4096
Policy	actor/critic MLP	[128, 128], ELU
RL algorithm	PPO	RSL-RL
Observation	per-step features	15
Observation	default history	$H = 20$
Observation	default dimension	300
Action	residual effort scale	2.8
Reward	position/velocity weights	4.0 / 0.5
Reward	smoothness/effort weights	0.02 / 0.001

Several implementation choices are important for reading the results. The residual effort scale is fixed at 2.8, so the normalized action RMS can be interpreted as how strongly the policy intervenes relative to its allowed range. The large number of parallel environments makes replay training efficient, but it also means that the policy sees many short transition windows rather than one continuous physical trial. This is appropriate for actuator calibration, but it makes the final benchmark a replay test, not a full robot deployment test.

The MLP architecture is intentionally small. A large network might fit the recorded trajectories more closely, but the goal is a compact correction layer that can later run alongside a locomotion simulator. The history window is therefore the main source of temporal modeling capacity. Experiment 1 directly tests whether increasing that temporal context helps or hurts the deployable UAN.

5 Datasets and Evaluation Metrics

The hardware CSV logs store time, current joint position, current joint velocity, command, next joint position, and next joint velocity:

$$t, q_{0:2}, \dot{q}_{0:2}, cmd_{0:2}, q_{next,0:2}, \dot{q}_{next,0:2}.$$

The new folder also contains data subsets for sample-efficiency experiments under `real_data/exp2_subsets`. These subsets are used to train deployable UAN models with different fractions of the available data.

5.1 Trajectory Families

The evaluation uses both walking-style trajectories and waveform trajectories. The walking-style logs are closest to the intended robot use case because they resemble leg motion during gait replay. The waveform logs are useful because they stress different actuator regimes: smooth periodic motion, abrupt changes, frequency changes, and non-sinusoidal ramps.

Table 5: Role of each dataset family in the deployable evaluation.

Dataset family	Main characteristic	Why it is useful
walk-air	walking-style motion without contact load	Tests actuator correction on gait-like trajectories.
walk-contact	walking-style motion with foot-ground interaction	Tests whether correction remains useful under contact disturbance.
sine	smooth periodic command	Clean training/evaluation signal for reusable actuator dynamics.
Gaussian	smooth random-like variation	Checks robustness outside a single periodic waveform.
square chirp	abrupt command discontinuities changing frequency	Stresses delay, acceleration, and transient response. Tests whether correction works across motion speeds.
triangle	piecewise linear command	Tests ramps and velocity sign changes.

Table 6: Final deployable experiments used in this report.

Experiment	Evaluation	Main question
Exp1	walk-air, $H = 1, 5, 10, 20, 40$	How much history is useful with deployable observations?
Exp2	walk-air, 5–100% data subsets	How does data amount affect correction?
Exp3	walk-air, final H20	Does the updated final deployable policy reduce sim-to-real replay gap?
Exp4	OOD waveforms	Does sine-only deployable UAN generalize?
Exp5	walk-contact, final H20	Does the updated final policy remain useful under contact?

The main reported metrics are mean joint-position RMSE, mean joint-velocity RMSE, and residual action RMS:

$$\text{RMSE}_q = \frac{1}{3} \sum_{j=1}^3 \sqrt{\frac{1}{N} \sum_i (q_{i,j}^{\text{real}} - q_{i,j}^{\text{sim}})^2},$$

$$\text{RMSE}_{\dot{q}} = \frac{1}{3} \sum_{j=1}^3 \sqrt{\frac{1}{N} \sum_i (\dot{q}_{i,j}^{\text{real}} - \dot{q}_{i,j}^{\text{sim}})^2}, \quad \text{RMS}_a = \frac{1}{3} \sum_{j=1}^3 \sqrt{\frac{1}{N} \sum_i a_{i,j}^2}.$$

The action RMS is not a performance metric by itself. The zero-residual baseline has action RMS equal to zero by definition. It is reported to show how much residual effort the UAN uses to achieve its tracking improvement.

5.2 Metric Interpretation

The primary metric is q RMSE because the main actuator-calibration objective is to make the simulated joint positions follow the measured hardware positions. A reduction from about 0.026–0.028 rad to about 0.011–0.016 rad is meaningful because the simulator is being corrected by a learned residual rather than by changing the recorded command. In the presentation, q RMSE should be described as the clearest evidence that the UAN is correcting actuator position behavior.

dq RMSE is more subtle. It measures dynamic agreement, but it is also sensitive to how velocity is measured or filtered in the hardware logs. If velocity is derived by numerical differentiation, small encoder noise and timing offsets can produce large apparent velocity error. For this reason, a higher dq RMSE does not fully invalidate the q RMSE improvement, but it does identify a real limitation: the current residual policy can improve one-step position while making simulated velocity less smooth or less aligned with measured velocity.

Action RMS is a cost-of-correction metric. A model that improves q RMSE with very high residual action may be forcing the simulator aggressively instead of learning a physically smooth actuator correction. Therefore, the best model is not simply the one with the lowest q RMSE. The best engineering trade-off should have low q RMSE, acceptable dq RMSE, and moderate residual effort.

6 Results

6.1 Experiment 1: History Length Ablation

Experiment 1 evaluates deployable UAN models with different history lengths on walk-air data. The zero-residual baseline has q RMSE 0.02659 rad and dq RMSE 0.17032 rad/s.

Table 7: Exp1 deployable UAN history ablation on walk-air.

H	q RMSE	dq RMSE	Action RMS	Interpretation
1	0.01457	0.19427	0.40489	lightest useful memory
5	0.01172	0.24187	0.42567	strong position correction
10	0.01096	0.25480	0.49159	best q RMSE
20	0.01106	0.23169	0.45826	close to H10 with lower dq/action
40	0.02181	0.28682	0.52287	too long; performance degrades

Exp1 Deployable UAN: Main Metrics vs History Length

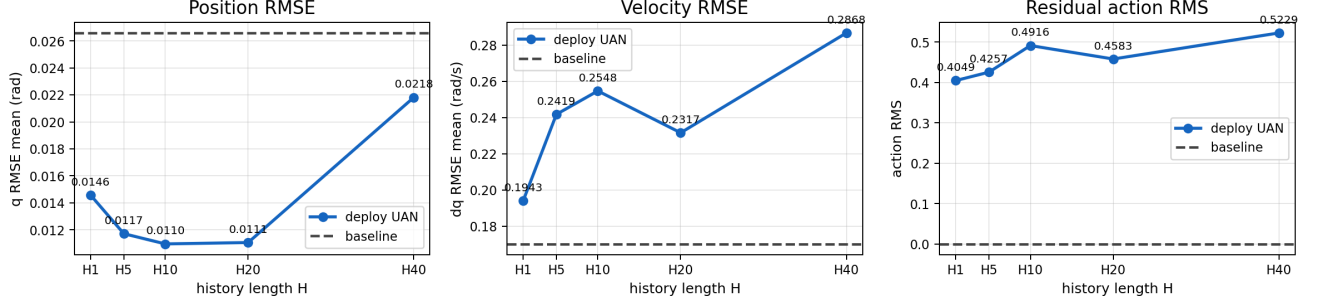


Figure 1: Exp1 main metrics. Deployable UAN reduces position RMSE for all history lengths, but velocity RMSE remains worse than baseline. H10/H20 are the best trade-off region.

The history-length result shows that more temporal context is not always better. Increasing history too much is harmful: $H = 40$ is much worse than $H = 10$ or $H = 20$. This likely happens because a long history creates a 600-dimensional observation and makes PPO harder to optimize. It may also include stale command and state information that is no longer useful for the current transition.

The best q RMSE occurs at $H = 10$, while $H = 20$ is almost equal in position error and better in velocity/action trade-off. This suggests that the useful actuator memory is present over a short time window, but the policy does not benefit from very long histories. In practical terms, $H = 10$ should be selected when the report emphasizes best position tracking, while $H = 20$ is a defensible engineering choice when slightly lower velocity error and lower residual action are also considered.

The $H = 1$ result is also informative. Even with almost no temporal history, q RMSE improves from 0.02659 rad to 0.01457 rad. This means the instantaneous command tracking error and simulator state already contain useful information for residual correction. However, adding moderate history improves q RMSE further, which supports the idea that actuator effects such as lag, friction, and short-term hysteresis are not completely memoryless.

6.2 Experiment 2: Sample Efficiency

Experiment 2 trains deployable UAN models using different data subset sizes and evaluates them on the same walk-air data.

Table 8: Exp2 sample efficiency on walk-air. Baseline q RMSE is 0.02659 rad.

Data subset	q RMSE	q improvement	dq RMSE	Action RMS
5%	0.01877	29.4%	0.25593	0.40067
10%	0.01681	36.8%	0.22101	0.41964
25%	0.02227	16.2%	0.21118	0.33827
50%	0.01264	52.5%	0.22130	0.42421
100%	0.01958	26.4%	0.25698	0.37053

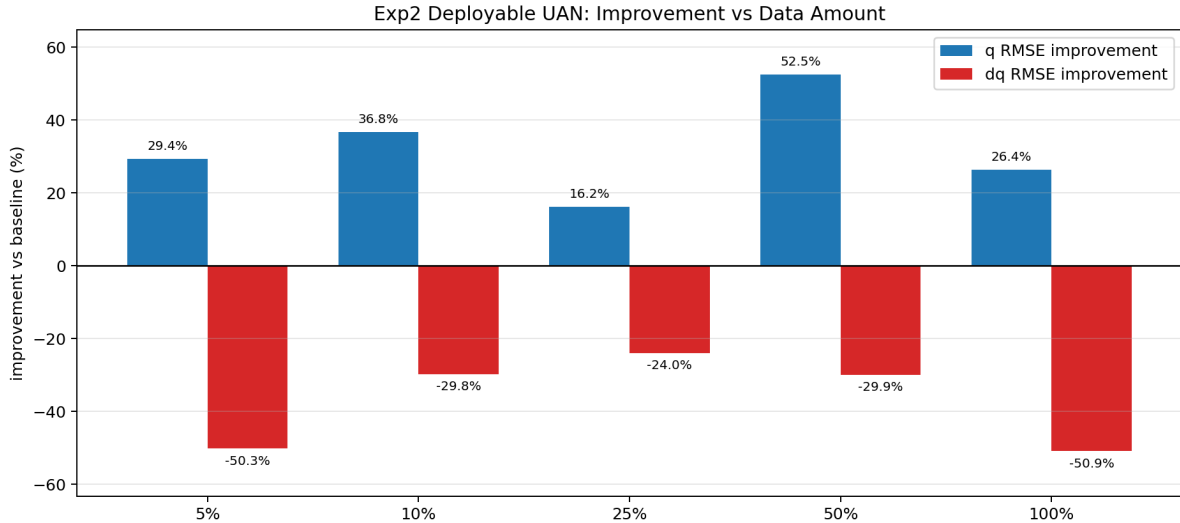


Figure 2: Exp2 sample efficiency. The relationship between data amount and performance is non-monotonic; the 50% subset performs best on walk-air.

The most important analysis point is that more data is not automatically better in this run. The 50% subset gives the best q RMSE, while the 100% model is worse. This does not mean that less data is fundamentally superior. A more likely explanation is that the subset composition, PPO stochasticity, and fixed training budget interact. Some subsets may contain more useful transition diversity, while the full dataset may include difficult or conflicting transitions without enough training time or reward shaping to fit them well. Therefore, Exp2 should be reported as early evidence of data sensitivity, not a final sample-efficiency law.

The non-monotonic result is important for the final discussion because it prevents an overly simple conclusion such as “more data always improves UAN.” The 5% and 10% models already improve q RMSE, which means the actuator mismatch contains learnable structure even in small subsets. The 25% model is worse than the 10% model, which suggests that subset identity matters. The 50% model gives the strongest result, possibly because it contains enough trajectory diversity while still avoiding some noisy or conflicting transitions. The 100% model may require longer training, different reward balancing, or stronger normalization to exploit the full dataset.

For a strict sample-efficiency claim, each subset size should be trained with multiple random seeds and evaluated on a frozen hold-out set. In this report, Exp2 should be interpreted as a practical engineering finding: deployable UAN can learn from partial data, but performance is sensitive to which transitions are included and how PPO optimization behaves.

6.3 Experiment 3: Sim-to-Real Gap on Walk-Air

Experiment 3 evaluates the updated final H20 deployable policy on the walk-air dataset. This experiment measures the sim-to-real replay gap under gait-like motion without foot contact.

Table 9: Exp3 final deployable UAN on walk-air.

Case	Baseline q	UAN q	q improvement	Baseline dq	UAN dq	Action RMS
H20 final	0.02673	0.01129	57.8%	0.16727	0.23064	0.45827

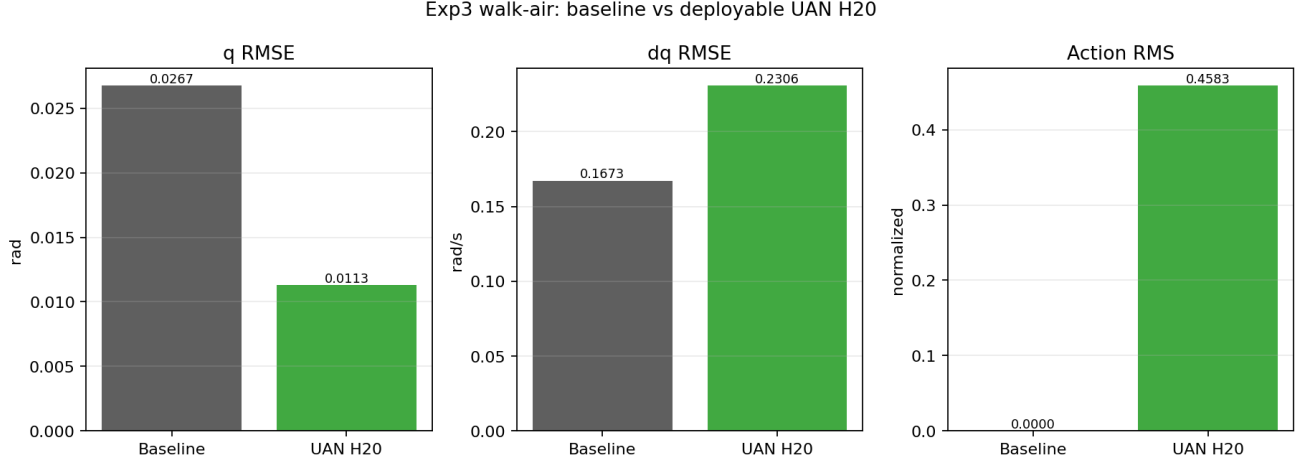


Figure 3: Exp3 walk-air grouped metrics. Deployable UAN reduces position RMSE, but velocity RMSE increases and residual action is required.

The deployable observation provides useful actuator correction from simulator-side signals alone. The updated final H20 policy reduces walk-air q RMSE by 57.8%, from 0.02673 rad to 0.01129 rad. This is a stronger final walk-air result than the earlier data-subset comparison and shows that the updated policy is a good candidate for the final actuator-position correction model.

The walk-air result is the cleanest demonstration of the main method because it uses a gait-like trajectory without the additional disturbance of foot contact. The position improvement is large enough to be visible in both aggregate metrics and time-series plots. At the same time, dq RMSE increases from 0.16727 rad/s to 0.23064 rad/s. This is the central trade-off of the current model: it is a useful position corrector, but it is not yet a complete dynamic actuator model.

Because this final Exp3 evaluation uses one H20 policy, it should be read as a final-policy replay result rather than a data-efficiency comparison. The data-efficiency question remains in Exp2, while Exp3 now answers whether the selected final deployable policy improves walk-air sim-to-real replay.

6.4 Experiment 4: Out-of-Distribution Generalization

Experiment 4 trains a sine-only deployable model and evaluates it on walk-air, Gaussian, square, chirp, and triangle datasets.

Table 10: Exp4 sine-only deployable UAN generalization.

Eval data	Baseline q	UAN q	q improvement	Baseline dq	UAN dq
walk-air	0.02659	0.02099	21.0%	0.17032	0.43564
Gaussian	0.02169	0.01883	13.2%	0.19208	0.34581
Square	0.07351	0.06637	9.7%	1.56917	1.51908
Chirp	0.03533	0.02259	36.1%	0.33945	0.46996
Triangle	0.02911	0.02040	29.9%	0.26620	0.42409

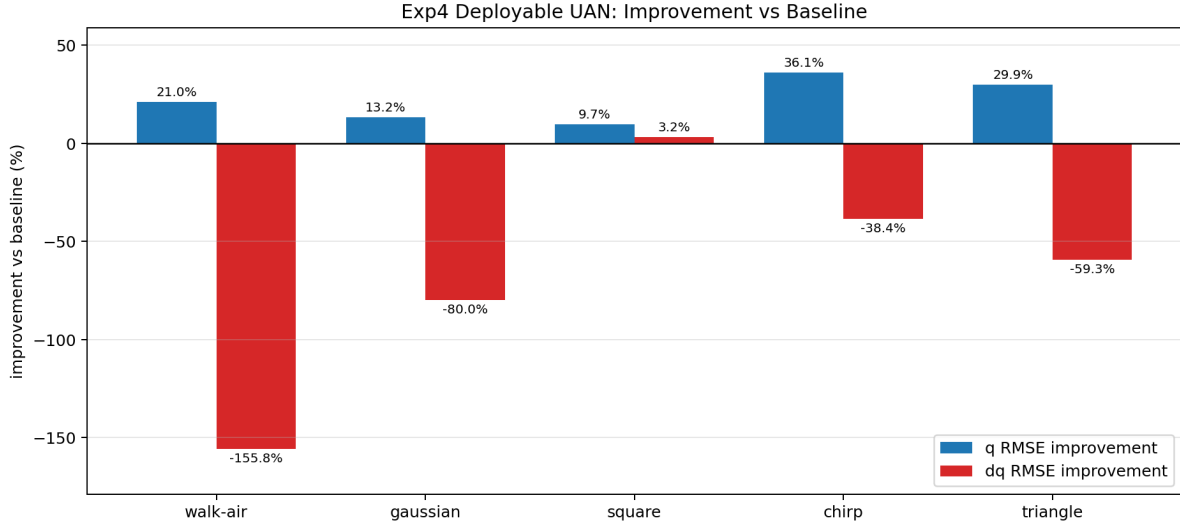


Figure 4: Exp4 improvement over baseline. The deployable sine-only model improves q RMSE on every dataset, but dq RMSE is worse except for a small square-wave improvement.

The sine-only model improves position RMSE on all evaluated datasets, which suggests that the learned residual captures a partially reusable actuator correction instead of memorizing only one waveform. The best OOD improvement is on chirp and triangle, while square remains difficult because it contains abrupt discontinuities. Velocity remains the main failure mode, with very large dq degradation on walk-air and Gaussian.

The strongest generalization percentage is on chirp, where q RMSE improves by 36.1%. This is useful because chirp changes frequency over time, so improvement there suggests that the residual correction is not restricted to one fixed motion speed. Triangle also improves by 29.9%, which indicates that ramp-like command segments are partially covered by the learned correction. Gaussian improves by 13.2%, a smaller but still positive result.

Square is the boundary case. It has the largest baseline q RMSE and dq RMSE, and the UAN only reduces q RMSE by 9.7%. This is expected because square commands contain abrupt discontinuities and high acceleration demands, while a sine-trained policy mainly sees smooth periodic motion. The square result should be presented as a useful negative case: the deployable UAN generalizes to several smooth or moderately changing trajectories, but discontinuous commands require explicit training coverage or stronger action regularization.

6.5 Experiment 5: Contact Stress Test

Experiment 5 evaluates the same updated final H20 deployable policy on walk-contact data. The baseline q RMSE is 0.02803 rad.

Table 11: Exp5 final deployable UAN on walk-contact.

Case	Baseline q	UAN q	q improvement	Baseline dq	UAN dq	Action RMS
H20 final	0.02803	0.01281	54.3%	0.18173	0.25073	0.45209

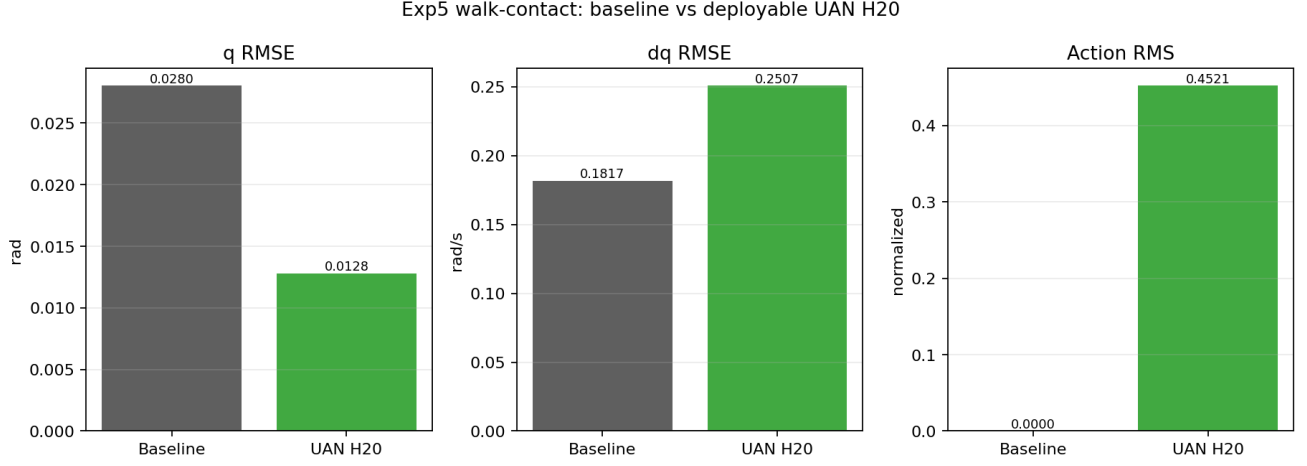


Figure 5: Exp5 contact metrics. Deployable UAN improves position tracking on contact data, but dq RMSE remains worse than baseline.

The contact result supports the usefulness of deployable UAN, but it also shows the current limitation. The final H20 policy reduces position RMSE by 54.3%, from 0.02803 rad to 0.01281 rad. However, velocity RMSE increases from 0.18173 rad/s to 0.25073 rad/s. The contact experiment should therefore be described as successful position correction under contact, not complete contact dynamics modeling.

Contact changes the interpretation of the residual action. In walk-air, residual effort mainly compensates for actuator lag and transmission effects during free motion. In walk-contact, the same residual must also respond to changes caused by foot-ground interaction. The fact that q RMSE still improves by more than 50% means the correction remains useful under a more realistic load condition. However, the velocity degradation shows that contact introduces dynamic effects that are not fully captured by the present training and reward setup.

The final contact policy uses action RMS 0.45209, close to the walk-air action RMS 0.45827. This reinforces the idea that residual action should not be interpreted alone. The best deployment criterion should combine q RMSE, dq RMSE, and residual smoothness rather than minimizing action RMS at the expense of tracking quality.

7 Overall Analysis

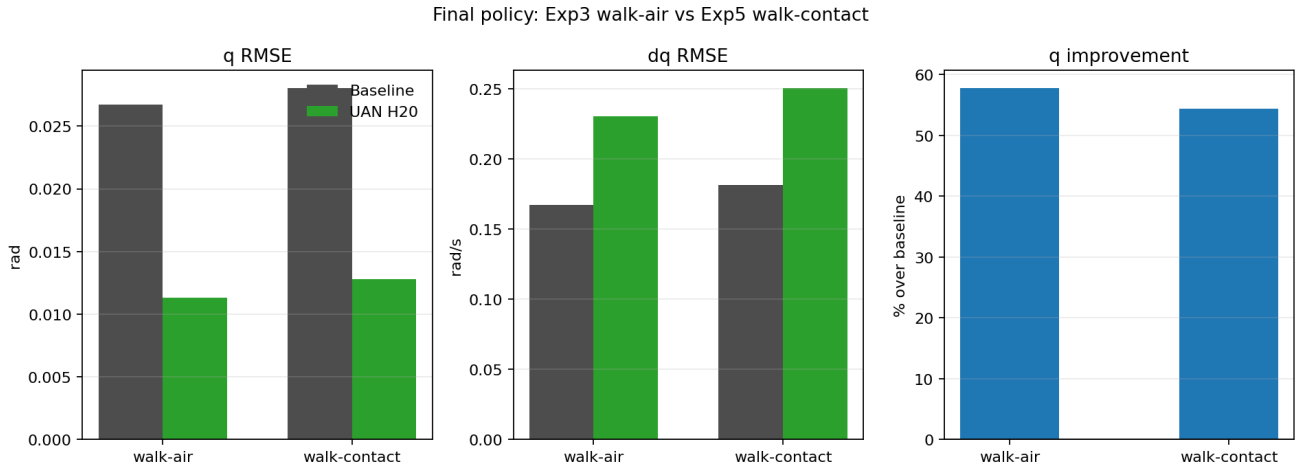


Figure 6: Summary of deployable UAN on walk-air and walk-contact. Position error improves, while velocity error and residual action remain the main trade-offs.

Across all deployable experiments, the consistent positive result is that UAN reduces joint-position RMSE. The best H10/H20 models in Exp1 reduce walk-air q RMSE to about 0.011 rad. The updated final Exp3 H20 policy reduces walk-air q RMSE to 0.01129 rad, and the updated final Exp5 H20 policy reduces walk-contact q RMSE to 0.01281 rad. These are meaningful improvements over the zero-residual baseline.

The main negative result is also consistent: velocity RMSE usually becomes worse. This happens for several reasons.

First, measured velocity may be noisier than position because it is often derived from encoder differentiation or filtering. Second, the reward emphasizes position more strongly than velocity. Third, residual effort can aggressively correct position while creating simulated velocity mismatch. Finally, the policy input is limited to command, simulator state, and previous action history, so the learned residual may improve the next position sample without producing physically smooth velocity.

The second important finding is that the deployable observation is useful but sensitive. It must infer a residual correction from command error, simulated state, command change, and previous action history. Therefore, the best result comes from a balance between enough temporal context and an observation that remains easy for PPO to optimize. This explains why H10/H20 work well while H40 becomes weaker.

The third finding is that data amount alone does not explain performance. Exp2 is non-monotonic: p50 is best, while p100 is worse. This suggests that data diversity, subset quality, training budget, and optimization noise matter. A final sample-efficiency study should repeat each subset with multiple seeds and freeze a validation/test split. The current Exp2 is useful evidence that UAN performance is sensitive to dataset composition, but it is not yet a definitive scaling law.

7.1 Why Position Improves More Than Velocity

The reward, data, and actuator behavior all favor position improvement. The position term has the largest weight and a tight scale, so PPO receives strong reward feedback when q error decreases. The velocity term is weaker and can be affected by measurement noise, filtering, and small time alignment errors. Therefore, a policy can learn residual actions that place the simulated joint closer to the next measured position while still producing velocity traces that do not match the measured velocity well.

This does not mean velocity can be ignored. For a future locomotion simulator, velocity mismatch can affect contact timing, swing-leg clearance, and stability. The current result should be interpreted as a successful first layer of actuator correction: the model learns useful position compensation, but the dynamic part of the correction needs more work. Better velocity smoothing, multi-step rewards, or a loss that emphasizes phase and smoothness may be needed.

7.2 Why Exp2 is Non-Monotonic

The p50 result in Exp2 is stronger than p100, even though p100 uses more data. A likely reason is that the p50 subset contains enough representative motion while producing an easier optimization problem. In contrast, the p100 dataset may contain more heterogeneous transitions, including difficult contact-like, high-acceleration, or noisy samples. With a fixed training budget and one PPO seed, more data can increase the learning burden instead of immediately improving generalization.

Another possible explanation is reward conflict. If different parts of the full dataset require different residual behavior, a single compact MLP may settle on a compromise policy that is not optimal for the walk-air evaluation. This is why the p100 result should not be described as proof that full data is bad. It is evidence that training protocol matters. A fair future comparison should increase training iterations for larger datasets, run multiple seeds, and evaluate on a fixed hold-out benchmark.

7.3 Threats to Validity

Several issues limit how strongly the results can be claimed. First, the contact experiment is still a replay evaluation. It uses recorded contact trajectories and compares one-step simulation transitions to hardware logs, but it is not the same as deploying the corrected actuator model inside a full closed-loop walking robot. Second, velocity labels may be less reliable than position labels, especially if they are derived from differentiated encoder signals.

Third, the zero-residual baseline is a useful simulator baseline, but it is not a manually tuned actuator model with identified friction, damping, latency, and motor constants. A stricter future benchmark should compare the deployable UAN against both the default simulator and a tuned analytical actuator model. Fourth, the sample-efficiency results were not repeated across random seeds, so the non-monotonic curve should be interpreted cautiously.

Finally, the one-leg rig removes full-body coupling. A hexapod has body inertia, six-leg coordination, contact timing, stance loading, and terrain interaction. The current result shows that actuator replay can be improved at the joint-transition level. It does not yet prove that a full locomotion policy trained with this actuator layer will transfer directly to hardware.

8 Code Reproducibility

The report is tied to generated artifacts in the deployable evaluation folder. The main scripts are `train_uan_deploy.py` for training, `evaluate_deploy_dataset.py` for replay evaluation, and `inspect_dataset.py` plus `plot_recorded_data.py` for dataset sanity checks. Each experiment folder under `eval_drl_experiments` stores the corresponding metric CSV, replay plot, and derived report figures. The key reproducibility rule is that baseline and UAN must be evaluated on the same replay starts, otherwise a lower RMSE could come from easier sampled windows rather than a better actuator correction.

9 Discussion: Implications for Hexapod Sim-to-Real

The practical value of the deployable UAN is that it can become an actuator layer inside future locomotion training. Instead of training a hexapod policy in a simulator with ideal joint response, the simulator can apply a learned residual correction that better matches the measured actuator behavior of the real robot. This should reduce the chance that the locomotion policy exploits unrealistically fast, smooth, or perfectly damped actuator dynamics.

The one-leg result is especially useful because it exposes the trade-offs before full locomotion makes them harder to diagnose. If a full hexapod policy fails, the cause may be gait reward design, contact modeling, terrain, controller frequency, motor behavior, or body dynamics. In the one-leg setup, the main question is narrower: does residual actuator correction make simulated joint transitions closer to measured hardware transitions? The answer from the current deployable experiments is yes for position, but not yet for velocity.

For the final presentation, the most honest story is therefore balanced. The project does not claim completed full hexapod walking. It claims a working deployable actuator-correction pipeline, evaluated across history length, data subset size, walk-air replay, OOD waveforms, and contact replay. The positive result is consistent q RMSE reduction. The negative result is velocity degradation and sensitivity to training data. That combination is a useful engineering outcome because it identifies exactly what must be improved before the UAN becomes a mature actuator model.

10 Limitations and Future Work

The current deployable UAN is a proof of concept rather than a final actuator model. The main limitations are:

- Velocity RMSE is worse in most experiments.
- Residual action RMS is nonzero and sometimes large.
- Exp2 is non-monotonic and was not repeated across random seeds.
- The contact test is still a replay evaluation, not full payload robustness.
- The system is still one-leg actuator calibration, not complete hexapod locomotion.

The next step is to improve both the experiment protocol and the model objective. The dataset should be split into fixed train, validation, and final hold-out test sets. Velocity should be smoothed or reweighted, and action-rate regularization should be increased to reduce aggressive residual effort. The most important model comparisons should include the zero-residual simulator baseline, deployable UAN trained on non-contact data, and deployable UAN trained with contact-rich data. After the deployable UAN is stable, it can be inserted into full hexapod locomotion training as a learned actuator correction layer.

11 Conclusion

The final project implements a deployable UAN actuator-correction pipeline for a 3-DOF one-leg rig. The policy observes command tracking error, simulated joint state, command change, and previous residual action, making it usable inside a future walking simulator.

The new deployable results still show clear position-tracking improvement. The updated final walk-air evaluation reduces q RMSE from 0.02673 rad to 0.01129 rad, and the updated final walk-contact evaluation reduces q RMSE from 0.02803 rad to 0.01281 rad. A sine-only deployable model also improves q RMSE on several out-of-distribution waveform families. However, velocity RMSE usually increases, and the sample-efficiency result is non-monotonic. Therefore, the correct final claim is: deployable UAN is a successful proof of concept for actuator position correction, but further work is required before it becomes a complete dynamics model for full hexapod sim-to-real transfer.